Universidad Rafael Landivar
Campus San Alberto Hurtado S.J.
Facultad de Ingeniería
Manejo e Implementación de Archivos
Laboratorio 2

Agenda de Contactos con Acceso Aleatorio

Keila Belinda Puac Gómez 1660625
Morelia González Ixcot 1536325
Jackelin Sofía Vásquez Guzman 1503725

Fecha: 28 de octubre de 2026

1. Resumen del proyecto

La aplicación administra una agenda de contactos guardada en un archivo binario de **registros de tamaño fijo** (`contactos.dat`). Gracias al tamaño fijo, cualquier contacto se localiza con acceso aleatorio mediante `seek()`, sin recorrer el archivo. Un índice primario (`indice.json`) relaciona cada ID con su número de registro. La interfaz está hecha con PyQt6.

Esta tercera parte agrega seguridad e información del sistema de archivos: verificación de integridad con SHA-256, respaldos comprimidos y consulta de metadatos. Módulos del proyecto:

Módulo	Responsabilidad
manejo_archivos.py	Registros de 100 bytes, UTF-8, seek() / tell(), alta, lectura y modificación en el lugar.
indice.py	Índice primario {ID → número de registro} en indice.json, con caché en memoria.
seguridad.py	Parte 3: hash SHA-256 (hashlib), respaldo ZIP (zipfile), metadatos (os.stat / os.access).
exportacion.py	Exportación a JSON y reporte PDF (parte 2).
interfaz.py	Ventana principal PyQt6; menú Herramientas con integridad, respaldo e información.

2. Estructura de campos y tamaños

Cada contacto ocupa exactamente **100 bytes**. Cada campo se codifica en UTF-8, se recorta a su longitud *en bytes* sin partir un carácter y se rellena con espacios a la derecha. Los campos se guardan uno tras otro, sin separadores:

Campo	Tamaño (bytes)	Posición inicial (offset)	Posición final	Contenido
id	5	0	4	Número entero positivo (clave primaria)
nombre	30	5	34	Nombre completo
telefono	15	35	49	Teléfono
correo	30	50	79	Correo electrónico
ciudad	20	80	99	Ciudad
Total	100			TAM_REGISTRO = 100

```
# manejo_archivos.py
TAM_REGISTRO = 100            # tamaño fijo en bytes
L_ID, L_NOMBRE, L_TEL, L_CORREO, L_CIUDAD = 5, 30, 15, 30, 20   # suman 100

def _campo(valor, longitud):
    """Codifica en UTF-8, recorta a -longitud- bytes sin partir un carácter y rellena con espacios."""
    datos = str(valor).encode("utf-8")[:longitud]
    datos = datos.decode("utf-8", errors="ignore").encode("utf-8")
    return datos.ljust(longitud, b" ")
```

Consecuencia del tamaño fijo: el registro número *n* (empezando en 0) siempre comienza en el byte `n × 100`. El tamaño del archivo es siempre múltiplo de 100, y por eso `total_registros() = tamaño // 100`. Al iniciar, la aplicación comprueba que `tamaño % 100 == 0`; si no, avisa que el último registro está incompleto.

Nota sobre UTF-8: las letras con tilde y la «ñ» ocupan 2 bytes, por eso los límites se validan en bytes y no en caracteres (por ejemplo, «Quetzaltenango» ocupa 14 bytes de los 20 disponibles, mientras que «Totonicapán» ocupa 12).

3. Funcionamiento técnico de seek() y tell()

Un archivo abierto mantiene un **puntero de posición**: el byte desde el cual se hará la siguiente lectura o escritura.

Método	Qué hace
f.tell()	Devuelve la posición actual del puntero, en bytes desde el inicio del archivo.
f.seek(desplazamiento, origen)	Mueve el puntero. Origen `os.SEEK_SET` (0): desde el inicio; `os.SEEK_CUR` (1): desde la posición actual; `os.SEEK_END` (2): desde el final.

Ambos métodos requieren abrir el archivo en **modo binario** (`'rb'`, `'ab'`, `'r+b'`), porque en modo texto las posiciones no equivalen a bytes cuando hay UTF-8 de varios bytes.

3.1 Agregar al final: seek(0, SEEK_END) y tell()

Se mueve el puntero al final; `tell()` da el byte donde empezará el nuevo registro, y de ahí se obtiene su número: `pos_actual // 100`.

```
with open(ARCHIVO_DAT, "ab") as f:          # "ab" crea el archivo si no existe
    f.seek(0, os.SEEK_END)                  # puntero al final
    pos_actual = f.tell()                   # p. ej. 1100 si ya hay 11 registros
    f.write(registro)                       # escribe 100 bytes
num_registro = pos_actual // TAM_REGISTRO   # 1100 // 100 = 11
```

3.2 Lectura directa (acceso aleatorio): seek(n × 100)

```
with open(ARCHIVO_DAT, "rb") as f:
    f.seek(num_registro * TAM_REGISTRO, os.SEEK_SET)   # salta directo al registro
    posicion = f.tell()                                # confirma la posición
    datos = f.read(TAM_REGISTRO)                       # lee exactamente 100 bytes
```

3.3 Modificación en el lugar: abrir con 'r+b'

El modo `'r+b'` permite leer y escribir sin borrar el archivo. Se coloca el puntero al inicio del registro y se sobrescriben solo sus 100 bytes; el resto del archivo no se toca ni se reescribe.

```
with open(ARCHIVO_DAT, "r+b") as f:
    f.seek(posicion, os.SEEK_SET)           # posicion = num_registro * 100
    posicion = f.tell()
    f.write(registro)                       # sobrescribe solo ese registro
```

La aplicación muestra en pantalla el último acceso realizado. Ejemplos obtenidos durante las pruebas:

Operación	Resultado mostrado por la aplicación
Agregar el contacto #12 (12.º registro)	Agregar (al final): registro #11 · tell() = 1100 (seek(11 × 100))
Buscar el ID 5	Lectura por seek(): registro #4 · tell() = 400 (seek(4 × 100))
Modificar el ID 2	Modificación en el lugar: registro #1 · tell() = 100 (seek(1 × 100))

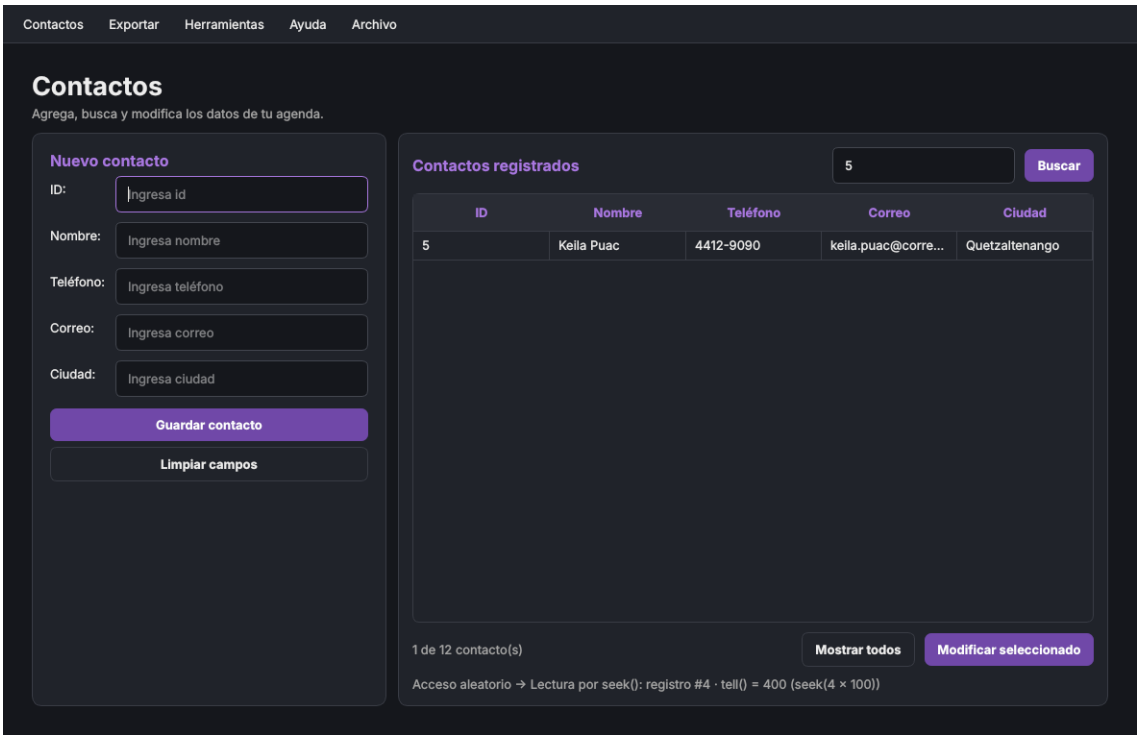


Figura 1. Búsqueda por ID: el índice da el registro y seek() salta directamente a su posición (línea inferior: «Acceso aleatorio → …»).

4. Justificación de la eficiencia del índice

Sin índice, encontrar un ID exige leer registros desde el inicio hasta dar con él: costo **O(n)**, y en el peor caso se leen los *n* × 100 bytes del archivo. **Con índice**, el ID se busca en un diccionario (tabla hash, costo **O(1)** promedio) que devuelve el número de registro, y se lee **un solo registro** con `seek(n × 100)`, también O(1). El costo de la lectura deja de depender de cuántos contactos existan.

Verificación experimental. Se midió, con los mismos módulos del proyecto, la búsqueda del *último* ID (peor caso secuencial) en archivos de 1 000, 10 000 y 50 000 registros (promedio de varias repeticiones):

Registros	Tamaño de contactos.dat	Secuencial (ms)	Índice releído de disco en cada búsqueda (ms)	Índice en memoria + seek() (ms) *
1 000	0.10 MB	0.28	0.24	0.021
10 000	1.00 MB	2.59	2.61	0.025
50 000	5.00 MB	12.31	16.55	0.030

** Versión entregada: indice.py conserva el índice en memoria y solo lo relee si indice.json cambia en disco (fecha de modificación o tamaño).*

Conclusiones de la medición. (1) El tiempo secuencial crece linealmente con el número de registros (de 0.28 ms a 12.3 ms), mientras que con el índice en memoria permanece prácticamente constante (≈ 0.03 ms con 50 000 registros). (2) Un índice que se vuelve a leer y analizar desde disco en cada consulta *no* es una mejora: en las pruebas resultó más lento que el barrido secuencial (16.6 ms contra 7.7 ms con 50 000 registros). Por eso se agregó el caché en memoria; la ventaja del índice proviene de consultarlo en memoria y de tocar el disco

únicamente para leer el registro buscado. (Las pruebas se hicieron con los archivos en la caché del sistema operativo; con disco real y archivos mayores la diferencia a favor del índice sería aún más marcada.)

Además, el índice ocupa poco espacio (≈ 20 bytes por entrada en el JSON frente a 100 bytes por registro), impide IDs duplicados (`agregar_al_indice` lanza error si la clave ya existe) y se reconstruye automáticamente desde `contactos.dat` si al iniciar no coincide con el archivo de datos.

5. Integridad de datos con SHA-256

SHA-256 es una función hash criptográfica que convierte cualquier contenido en una huella de 256 bits (64 caracteres hexadecimales). Cambiar un solo bit del archivo produce una huella completamente distinta, por lo que sirve para detectar modificaciones hechas fuera de la aplicación. La implementación está en `seguridad.py` con el módulo `hashlib`; el hash se guarda en `contactos.sha256`.

```
def calcular_hash(ruta=ARCHIVO_DATOS):
    h = hashlib.sha256()
    with open(ruta, "rb") as f:
        for bloque in iter(lambda: f.read(8192), b""):   # lectura por bloques de 8 KB
            h.update(bloque)
    return h.hexdigest()

def guardar_hash(ruta=ARCHIVO_DATOS, ruta_hash=ARCHIVO_HASH):
    with open(ruta_hash, "w", encoding="utf-8") as f:
        f.write(calcular_hash(ruta))
```

Cuándo se actualiza el hash. Después de cada escritura sobre contactos.dat, tanto al agregar como al modificar un contacto:

```
# interfaz.py → registrar_contacto() y modificar_contacto()
manejo_archivos.agregar_contacto(...)          # o modificar_contacto_en_lugar(...)
ok, mensaje = seguridad.guardar_hash()         # recalcula y guarda el SHA-256
if not ok:
    QMessageBox.warning(self, "Hash no actualizado", mensaje)
```

Momento	Hash SHA-256 de contactos.dat (12 contactos)
Después de agregar los 12 contactos	`dc2ffa0c8d63be73a9892bc2ab8e91e1777403511883f2e37f5e298c7ab3345e`
Después de modificar el teléfono del contacto #2 (cambia 1 registro)	`ffa684b1096ab5d6350dc17705089790f635d7a49c721835bae177ceda6bcd6e`

Verificación al iniciar. Al abrir la aplicación se recalcula el hash y se compara con el guardado. `verificar_integridad()` devuelve un estado que la interfaz traduce en un mensaje:

```
def verificar_integridad(ruta=ARCHIVO_DATOS, ruta_hash=ARCHIVO_HASH):
    ...
    if actual == guardado:
        return "ok", "Integridad verificada: el archivo no ha sido modificado."
    return "alterado", ("¡ADVERTENCIA! El archivo fue modificado fuera de la aplicación.\n\n"
                        "Guardado: %s\nActual:   %s" % (guardado, actual))
```

Estado	Situación	Reacción de la aplicación
ok	Hash actual = hash guardado	Silencioso al iniciar; mensaje de confirmación si el usuario lo pide.
alterado	Hash distinto (cambió el contenido o el tamaño)	Alerta con ambos hashes; el usuario decide si acepta el estado actual y regenera el hash.

Estado	Situación	Reacción de la aplicación
sin_hash	Existe contactos.dat pero no contactos.sha256	Genera el hash inicial.
sin_datos	Aún no existe contactos.dat	No hay nada que verificar.
error	Sin permisos o error de lectura	Mensaje de advertencia; la aplicación no se cae.

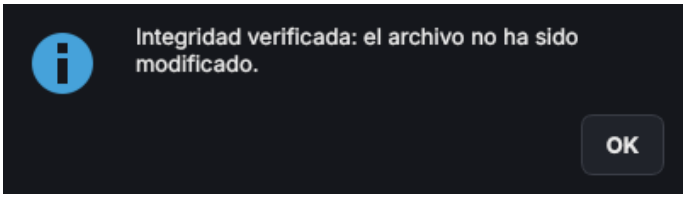


Figura 2. Verificación correcta: el archivo no ha sido modificado.

5.1 Prueba: alteración manual del archivo

Con la aplicación cerrada se modificó **un solo byte** de contactos.dat con un editor externo: el primer dígito del teléfono del registro #2 (offset 235) se cambió de «5» a «9» (de «5890-1122» a «9890-1122»). El archivo conservó su tamaño de 1 200 bytes. Al volver a abrir la aplicación apareció la alerta:

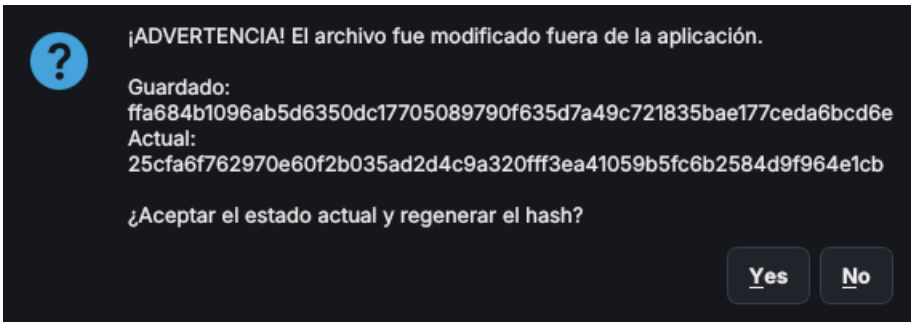


Figura 3. Alerta al iniciar: el archivo fue alterado fuera de la aplicación; se muestran el hash guardado y el actual.

	SHA-256
Hash guardado (antes de la alteración)	`ffa684b1096ab5d6350dc17705089790f635d7a49c721835bae177ceda6bcd6e`
Hash actual (1 byte alterado)	`25cfa6f762970e60f2b035ad2d4c9a320fff3ea41059b5fc6b2584d9f964e1cb`

Los dos hashes no se parecen en nada pese a que solo cambió un byte de 1 200: es el *efecto avalancha* de SHA-256. También se probó que se detecta una alteración que cambia el tamaño (se añadió un byte al final) y que, si falta contactos.sha256, la aplicación genera uno nuevo en lugar de fallar.

6. Respaldo comprimido (zipfile)

La opción **Herramientas → Crear respaldo ZIP** empaqueta `contactos.dat` e `indice.json` en un archivo `respaldo_AAAAMMDD_HHMMSS.zip` con compresión DEFLATE. Cada respaldo lleva fecha y hora en el nombre, de modo que no se sobrescriben. Después de crearlo se verifica con `testzip()` y se muestran las estadísticas de tamaño.

```
with zipfile.ZipFile(ruta_zip, "w", zipfile.ZIP_DEFLATED) as z:
    for a in existentes:
        z.write(a, arcname=os.path.basename(a))

with zipfile.ZipFile(ruta_zip, "r") as z:
    if z.testzip() is not None:                    # verifica la integridad del ZIP
        return False, "El respaldo se creó pero falló su verificación."
    detalle = [(i.filename, i.file_size, i.compress_size) for i in z.infolist()]
```

Archivo	Tamaño original (bytes)	Tamaño comprimido (bytes)	Reducción
contactos.dat	1200	508	57.7 %
indice.json	151	71	53.0 %
Total de datos	1351	579	57.1 %
Archivo .zip completo (incluye encabezados)	1351	801	40.7 %

Análisis. contactos.dat se comprime más de la mitad (1 200 → 508 bytes) porque sus registros están rellenos con espacios, que DEFLATE reduce muy bien. indice.json también se reduce por la repetición de su estructura. El ZIP final (801 bytes) es mayor que la suma de los datos comprimidos porque añade los encabezados y el directorio central de cada archivo; por eso la aplicación informa el tamaño real del .zip como cifra principal. Con archivos muy pequeños ese sobrecosto fijo pesa más; con agendas grandes el ahorro se acerca al de los datos comprimidos.

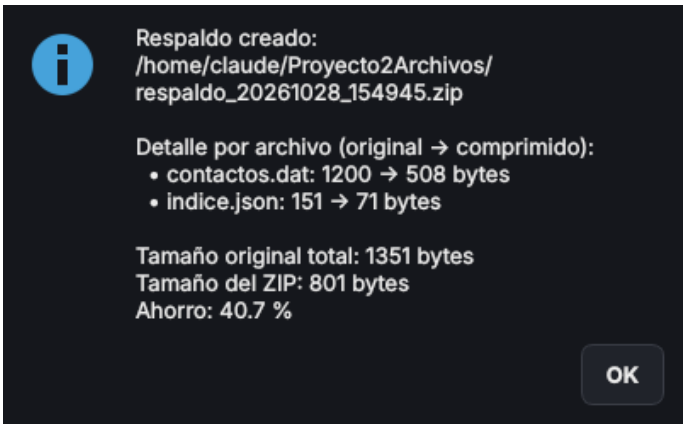


Figura 4. Ventana de estadísticas del respaldo: tamaño original vs. comprimido.

7. Metadatos del archivo (os.stat y os.access)

Herramientas → Información del archivo abre una ventana con los metadatos de contactos.dat. `os.stat()` aporta el tamaño (`st_size`) y la fecha de última modificación (`st_mtime`); `os.access()` indica si el proceso tiene permiso de lectura (`os.R_OK`) y escritura (`os.W_OK`).

```
def info_archivo(ruta=ARCHIVO_DATOS):
    st = os.stat(ruta)
    return True, {
        "ruta": os.path.abspath(ruta),
        "tamano_bytes": st.st_size,
        "modificado": datetime.fromtimestamp(st.st_mtime).strftime("%Y-%m-%d %H:%M:%S"),
        "lectura": os.access(ruta, os.R_OK),
        "escritura": os.access(ruta, os.W_OK),
```

```
    }
```



Figura 5. Ventana «Información del archivo»: tamaño, registros, última modificación y permisos.

Si el archivo no existe o no hay permisos, la función devuelve un mensaje y la interfaz lo muestra en un cuadro de aviso en lugar de cerrarse. La opción adicional **Ver metadatos** (página Herramientas) muestra lo mismo para contactos.dat, indice.json y contactos.sha256.

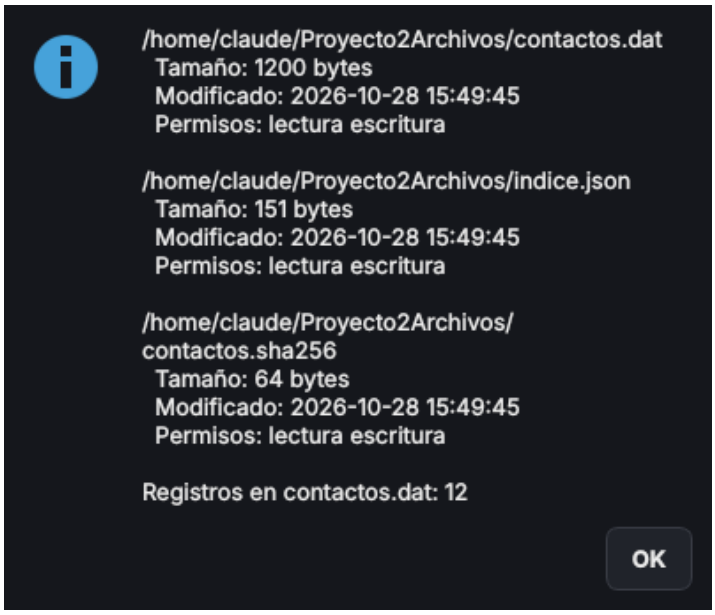


Figura 6. Metadatos de los tres archivos de datos de la aplicación.

8. Capturas de la aplicación funcionando

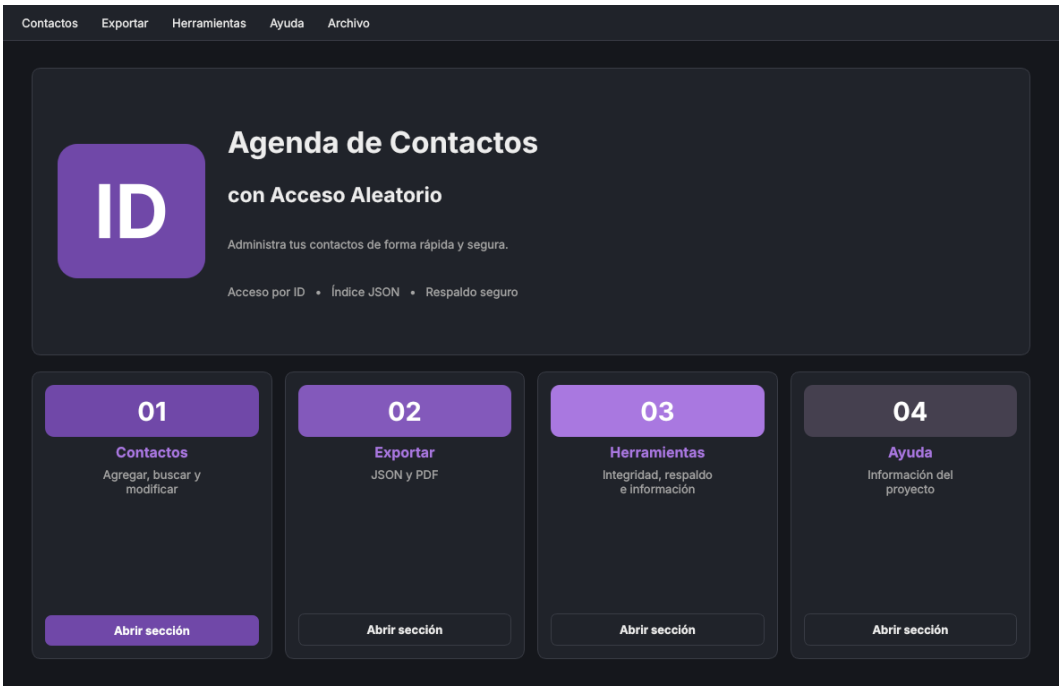


Figura 7. Pantalla de inicio.

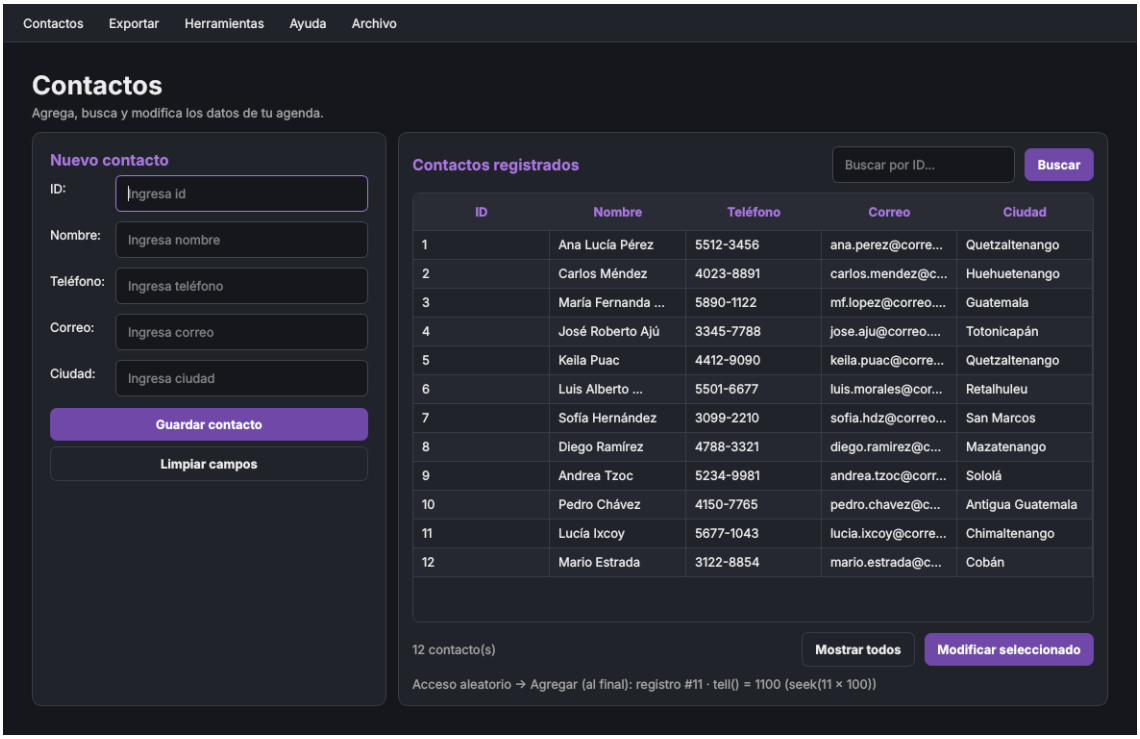


Figura 8. Contactos: alta de registros y tabla (12 contactos; la última línea muestra seek()/tell() del último acceso).

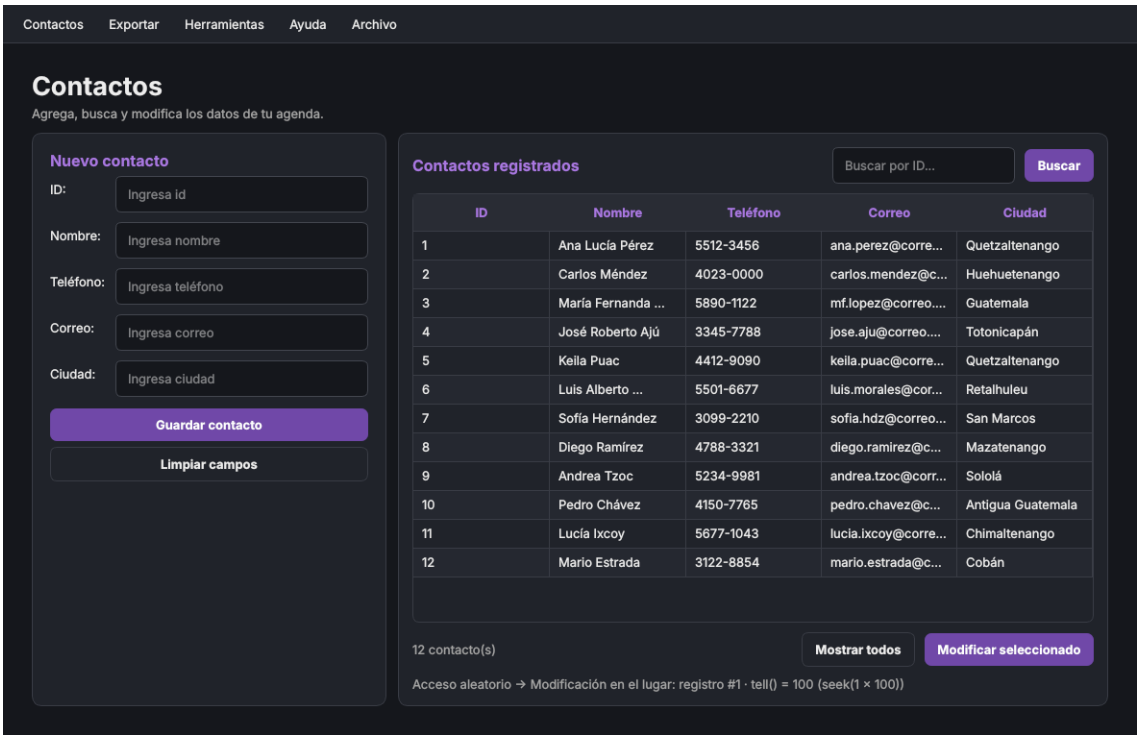


Figura 9. Tras modificar el contacto #2 en el lugar (solo se sobrescribieron sus 100 bytes).

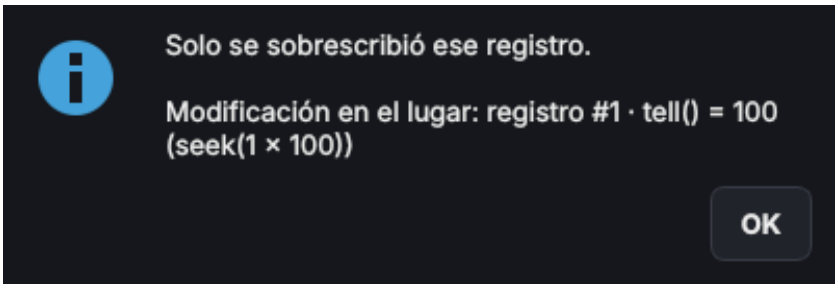


Figura 10. Confirmación de la modificación con la posición exacta de seek()/tell().

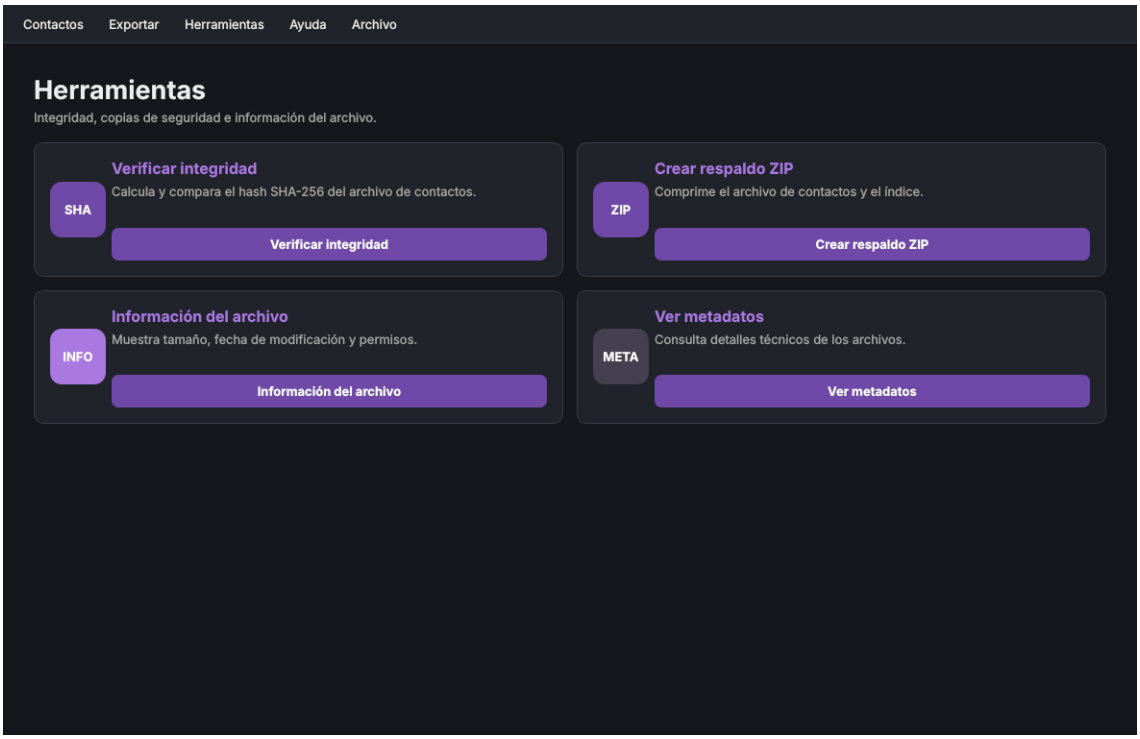


Figura 11. Página Herramientas: integridad, respaldo ZIP, información y metadatos.

9. Conclusiones

- El registro de tamaño fijo (100 bytes) convierte la ubicación de cualquier contacto en una operación aritmética ($n \times 100$), que seek() y tell() aprovechan para leer o sobrescribir un registro sin recorrer ni reescribir el archivo.
- El índice solo mejora el rendimiento si se consulta en memoria: se verificó que releerlo de disco en cada búsqueda era más lento que el barrido secuencial, y con el caché la búsqueda es prácticamente de tiempo constante.
- SHA-256 detecta cualquier alteración externa de contactos.dat, incluso de un solo byte, y la aplicación advierte al usuario al iniciar. Es detección, no prevención: quien edite el archivo y también contactos.sha256 no sería detectado; para ello haría falta una firma con clave secreta (HMAC).
- El respaldo ZIP reduce el espacio de los datos y se verifica al crearse; los metadatos con os.stat y os.access dan visibilidad del tamaño, la fecha y los permisos del archivo.

Anexo: ejecución

```
python -m pip install PyQt6 fpdf2
python interfaz.py        # los archivos de datos se crean junto al programa
```